

UNIT - V DATA ON EXTERNAL STORAGE

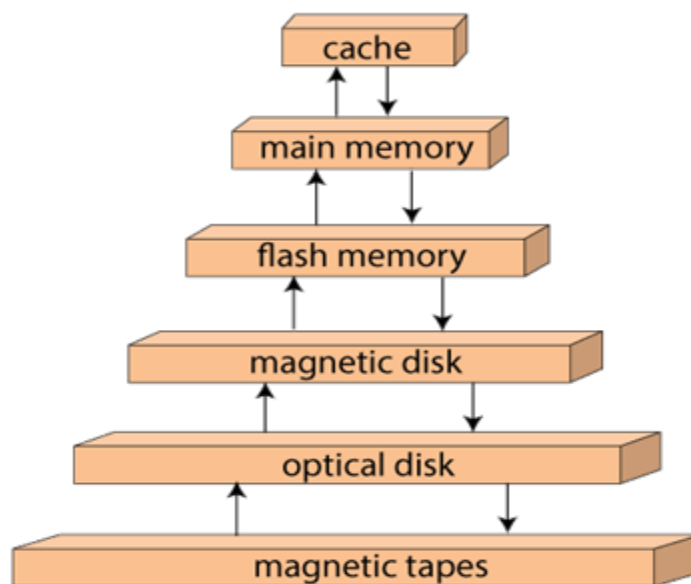
Data on External Storage, File Organization and Indexing, Cluster Indexes, Primary and Secondary Indexes, Index data Structures, Hash Based Indexing, Tree based Indexing, Comparison of File Organizations, Indexes- Intuitions for tree Indexes, Indexed Sequential Access Methods (ISAM), B+ Trees: A Dynamic Index Structure

Data on External Storage in DBMS

The storage system in a DBMS refers to the hierarchical arrangement of storage devices and media to store, manage, and retrieve data efficiently. The system is designed to handle different storage capacities, access speeds, volatility, and costs.

Storage System Hierarchy in DBMS

The storage hierarchy typically has multiple levels, each with its specific characteristics. Here's a typical hierarchy from fastest (and usually most expensive per byte) to slowest (and usually least expensive per byte):



Storage device hierarchy

1. Registers

- Located within the CPU.
- Smallest and fastest type of storage.
- Used to hold data currently being processed.

2. Cache Memory (L1, L2, L3 caches)

- On or very close to the CPU.
- Extremely fast but small in size.
- Acts as a buffer for frequently used data.

3. Main Memory (RAM)

- Data that's actively being used or processed is loaded here.
- Faster than secondary storage.
- Volatile in nature (i.e., data is lost when power is turned off).

4. Flash Storage (Solid State Drives - SSD)

- No moving parts.
- Faster than traditional hard drives.
- More durable and reliable.

5. Magnetic Disks (Hard Disk Drives - HDD)

- Primary secondary storage medium.
- Non-volatile, persistent storage.
- Data is stored in tracks, sectors, and cylinders.
- Slower than main memory but offers a large storage capacity at a lower cost.

6. Optical Disks (CD, DVD, Blu-Ray)

- Data is read using lasers.
- Slower than magnetic disks and usually have less storage capacity.
- Portable and commonly used for media and software distribution.

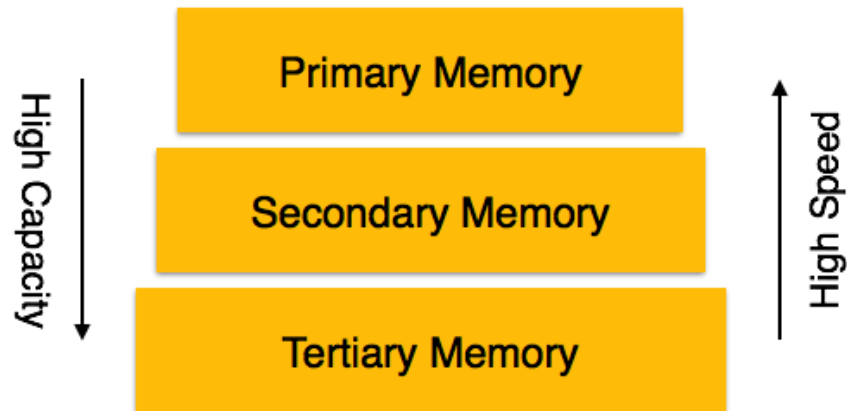
7. Magnetic Tapes

- Sequential access storage, unlike disks which are random access.
- Often used for backups and archiving due to their high capacity and low cost.
- Much slower access times compared to magnetic disks.

8. Remote Storage/Cloud Storage

- Data stored in remote servers and accessed over the internet.
- Provides scalability, availability, and fault-tolerance.
- Latency depends on network speed and distance to servers.

Types of Storage:



1. Primary Storage:

Includes registers, cache memory, and main memory (RAM). It's the main memory where the operating system, application programs, and data in current use are kept for quick access by the computer's processor.

2. Secondary Storage:

Encompasses data storage devices like HDDs, SSDs, CDs, and USB drives. It is non-volatile and retains data even when the computer is turned off.

3. Tertiary Storage or Off-line Storage:

Often involves magnetic tape systems or optical disk archives. This is slower than secondary storage and is used for data archiving and backup.

4. Quaternary Storage:

Refers to methods like cloud storage or other remote storage techniques where data is stored in remote servers and is fetched over the internet or other networks.

File Organization and Indexing

File organization and indexing are fundamental concepts in database management systems (DBMS) that deal with efficient storage, retrieval, and management of data.

File Organization

File organization refers to the arrangement of data on storage devices. The method chosen can have a profound effect on the efficiency of various database operations.

What is a File?

A file is named a collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tapes, and optical disks.

What is File Organization?

File Organization refers to the logical relationships among various records that constitute the file, particularly with respect to the means of identification and access to any specific record. In simple terms, Storing the files in a certain order is called File Organization. **File Structure** refers to the format of the label and data blocks and of any logical control record.

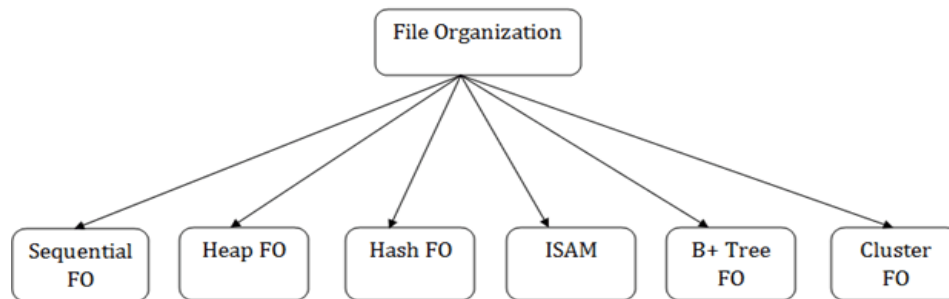
The Objective of File Organization

- It helps in the faster selection of records i.e. it makes the process faster.
- Different Operations like inserting, deleting, and updating different records are faster and easier.
- It prevents us from inserting duplicate records via various operations.
- It helps in storing the records or the data very efficiently at a minimal cost

Types of File Organizations

Various methods have been introduced to Organize files. These particular methods have advantages and disadvantages on the basis of access or selection. Thus it is all upon the programmer to decide the best-suited file Organization method according to his requirements.

Types of file organization are as follows:

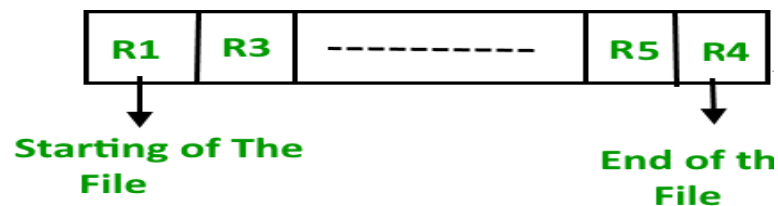


Sequential File Organization

The easiest method for file Organization is the Sequential method. In this method, the file is stored one after another in a sequential manner. There are two ways to implement this method:

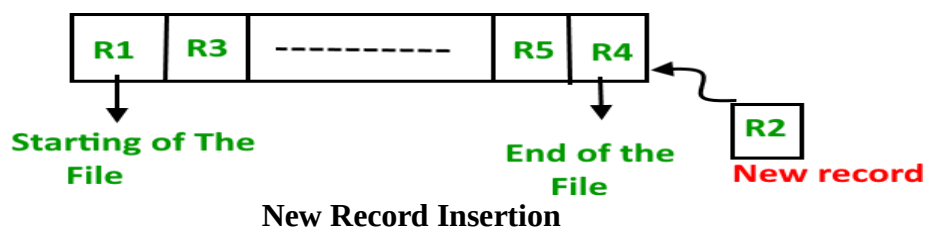
1. Pile File Method

This method is quite simple, in which we store the records in a sequence i.e. one after the other in the order in which they are inserted into the tables.



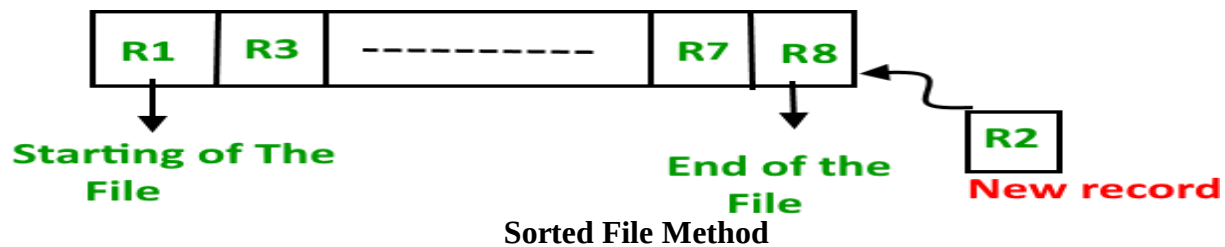
Pile File Method

Insertion of the new record: Let the R1, R3, and so on up to R5 and R4 be four records in the sequence. Here, records are nothing but a row in any table. Suppose a new record R2 has to be inserted in the sequence, then it is simply placed at the end of the file.

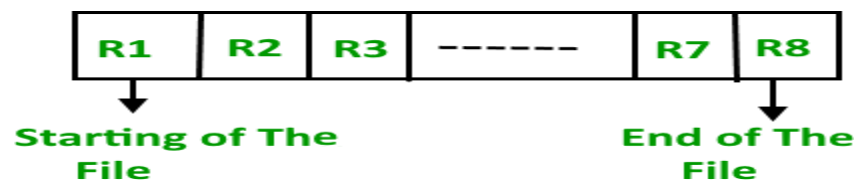


2. Sorted File Method

In this method, As the name itself suggests whenever a new record has to be inserted, it is always inserted in a sorted (ascending or descending) manner. The sorting of records may be based on any [primary key](#) or any other key.



Insertion of the new record: Let us assume that there is a preexisting sorted sequence of four records R1, R3, and so on up to R7 and R8. Suppose a new record R2 has to be inserted in the sequence, then it will be inserted at the end of the file and then it will sort the sequence.



Advantages of Sequential File Organization

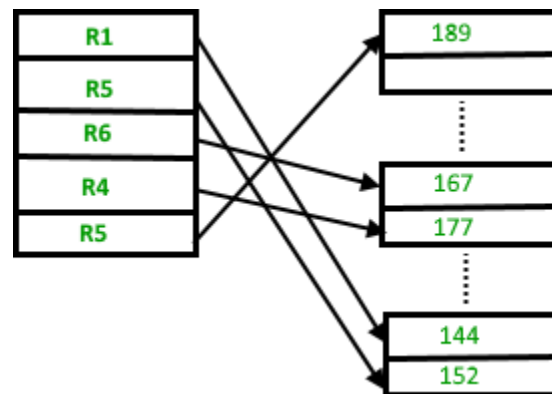
- Fast and efficient method for huge amounts of data.
- Simple design.
- Files can be easily stored in magnetic tapes i.e. cheaper storage mechanism.

Disadvantages of Sequential File Organization

- Time wastage as we cannot jump on a particular record that is required, but we have to move in a sequential manner which takes our time.
- The sorted file method is inefficient as it takes time and space for sorting records.

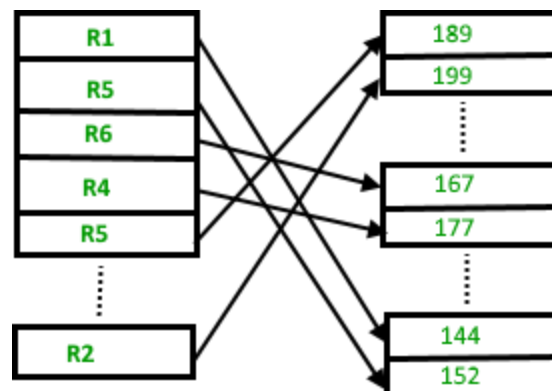
Heap File Organization

Heap File Organization works with data blocks. In this method, records are inserted at the end of the file, into the data blocks. No Sorting or Ordering is required in this method. If a data block is full, the new record is stored in some other block, Here the other data block need not be the very next data block, but it can be any block in the memory. It is the responsibility of DBMS to store and manage the new records.



Heap File Organization

Insertion of the new record: Suppose we have four records in the heap R1, R5, R6, R4, and R3, and suppose a new record R2 has to be inserted in the heap then, since the last data block i.e data block 3 is full it will be inserted in any of the data blocks selected by the DBMS, let's say data block 1.



New Record Insertion

If we want to search, delete or update data in the heap file Organization we will traverse the data from the beginning of the file till we get the requested record. Thus if the database is very huge, searching, deleting, or updating the record will take a lot of time.

Advantages of Heap File Organization

- Fetching and retrieving records is faster than sequential records but only in the case of small databases.
- When there is a huge number of data that needs to be loaded into the database at a time, then this method of file Organization is best suited.

Disadvantages of Heap File Organization

- The problem of unused memory blocks.
- Inefficient for larger databases.

3.Hash File Organization:

Hash File Organization uses Hash function computation on some fields of the records. The output of the hash function determines the location of disk block where the records are to be placed.

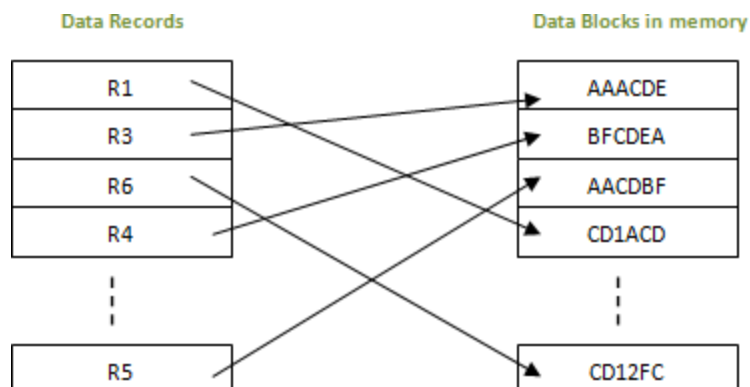
In this method of file organization, hash function is used to calculate the address of the block to store the records.

The hash function can be any simple or complex mathematical function.

The hash function is applied on some columns/attributes – either key or non-key columns to get the block address.

Hence each record is stored randomly irrespective of the order they come. Hence this method is also known as Direct or Random file organization.

If the hash function is generated on key column, then that column is called hash key, and if hash function is generated on non-key column, then the column is hash column.



When a record has to be retrieved, based on the hash key column, the address is generated and directly from that address whole record is retrieved. Here no effort to traverse through whole file. Similarly when a new record has to be inserted, the address is generated by hash key and record is directly inserted. Same is the case with update and delete.

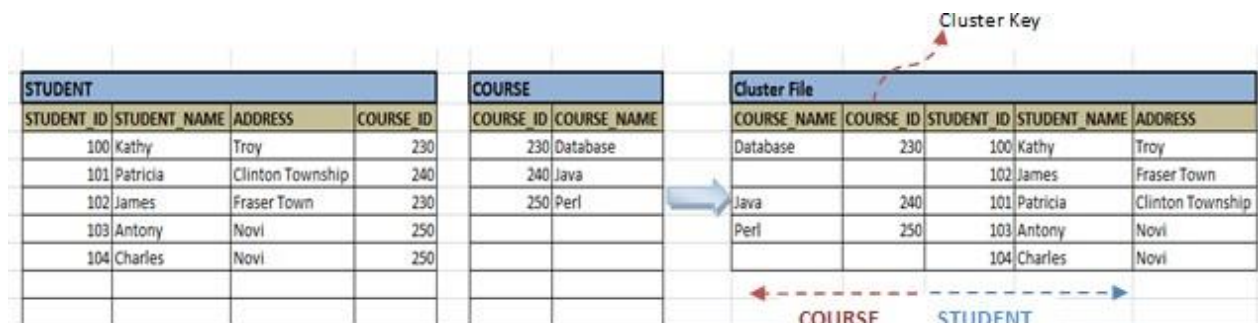
Advantages of Hash File Organization

- Records need not be sorted after any of the transaction. Hence the effort of sorting is reduced in this method.
- Since block address is known by hash function, accessing any record is very faster. Similarly updating or deleting a record is also very quick.
- This method can handle multiple transactions as each record is independent of other. i.e.; since there is no dependency on storage location for each record, multiple records can be accessed at the same time.
- It is suitable for online transaction systems like online banking, ticket booking system etc.

Clustered file organization:

Clustered file organization is not considered good for large databases. In this mechanism, related records from one or more relations are kept in the same disk block, that is, the ordering of records is not based on primary key or search key.

In this method two or more table which are frequently used to join and get the results are stored in the same file called clusters. These files will have two or more tables in the same data block and the key columns which map these tables are stored only once. This method hence reduces the cost of searching for various records in different files. All the records are found at one place and hence making search efficient.



B+ Tree File Organization

As its name implies, B+ Tree uses a tree-like structure to store records in a file. According to the key indexing principle, the primary key is used to sort the records. A produced index value is associated with the record for each primary key. An index of a record is a record's file address.

With the exception of having more children than just two, B+ Tree and binary search tree are extremely similar. The leaf nodes are where all the information is kept, while the intermediate nodes serve as pointers to those nodes. A sorted sequential linked list represents the data in leaf nodes at all times. Let us understand its advantages and disadvantages.

Advantages

- It's quicker and simpler to traverse a tree.
- All records are stored only in leaf nodes and are arranged in a sequential linked list makes searching simple.
- The size of a B+ tree is not constrained. As the size of the data changes, it might expand or contract.

Disadvantages

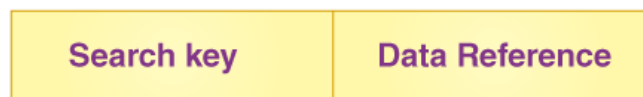
The only disadvantage of using this method is for static tables. It is ineffective.

Indexing

Indexing involves creating an auxiliary structure (an index) to improve data retrieval times. Just like the index in the back of a book, a database index provides pointers to the locations of records.

Structure of Index

We can create indices using some columns of the database.



Structure of Index

- The search key is the database's first column, and it contains a duplicate or copy of the table's candidate key or primary key. The primary key values are saved in sorted order so that the related data can be quickly accessible.
- The data reference is the database's second column. It contains a group of pointers that point to the disk block where the value of a specific key can be found.

Types of Indexes:

1. **Single-level Index:** A single index table that contains pointers to the actual data records.
2. **Multi-level Index:** An index of indexes. This hierarchical approach reduces the number of accesses (disk I/O operations) required to find an entry.
3. **Dense and Sparse Indexes:**
 - In a dense index, there's an index entry for every search key value in the database.



UP	→	UP	Agra	1,604,300
USA	→	USA	Chicago	2,789,378
Nepal	→	Nepal	Kathmandu	1,456,634
UK	→	UK	Cambridge	1,360,364

- In a sparse index, there are fewer index entries. One entry might point to several records.

China	→	China	Beijing	3,705,386
Russia	→	Canada	Ottawa	3,855,081
USA	→	Russia	Moscow	6,592,735
	→	USA	Washington	3,718,691

4. **Primary and Secondary Indexes:**

- A primary index is an ordered file whose records are of fixed length with two fields. The first field is the same as the primary key, and the second field is a pointer to the data block. There's a one-to-one relationship between the number of entries in the index and the number of records in the main file.
- A secondary index provides a secondary means of accessing data. For each secondary key value, the index points to all the records with that key value.

5. **Clustered vs. Non-clustered Index:**

- In a clustered index, the rows of data in the table are stored on disk in the same order as the index. There can only be one clustered index per table.
- In a non-clustered index, the order of rows does not match the index's order. You can have multiple non-clustered indexes.

6. **Bitmap Index:** Used mainly for data warehousing setups, a bitmap index uses bit arrays (bitmaps) and usually involves columns that have a limited number of distinct values.

7. **B-trees and B+ trees:** Balanced tree structures that ensure logarithmic access time. B+ trees are particularly popular in DBMS for their efficiency in disk I/O operations.

Benefits of Indexing:

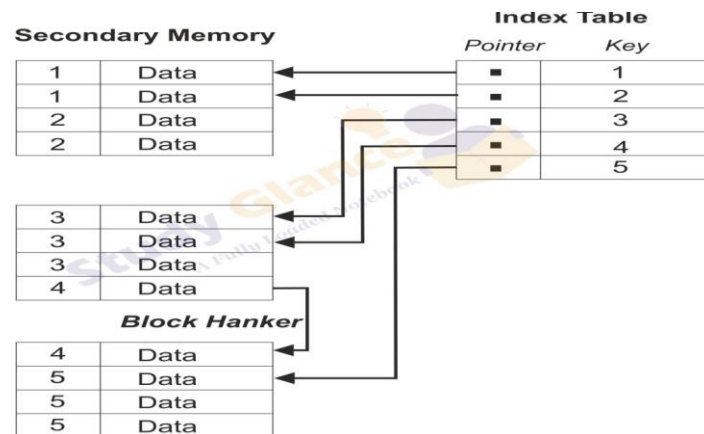
- Faster search and retrieval times for database operations.

Drawbacks of Indexing:

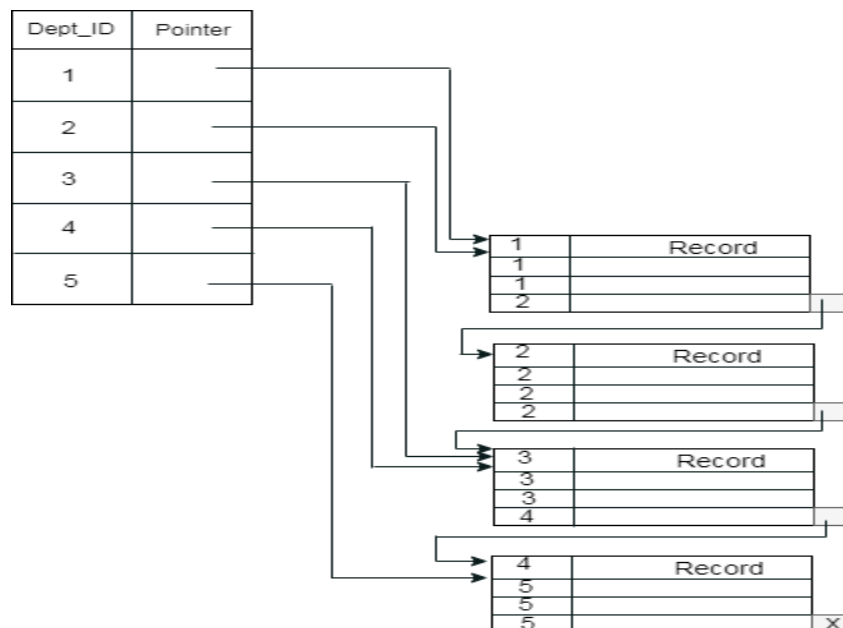
- Overhead for insert, update, and delete operations, as indexes need to be maintained.
- Additional storage requirements for the index structures.

Cluster Indexes

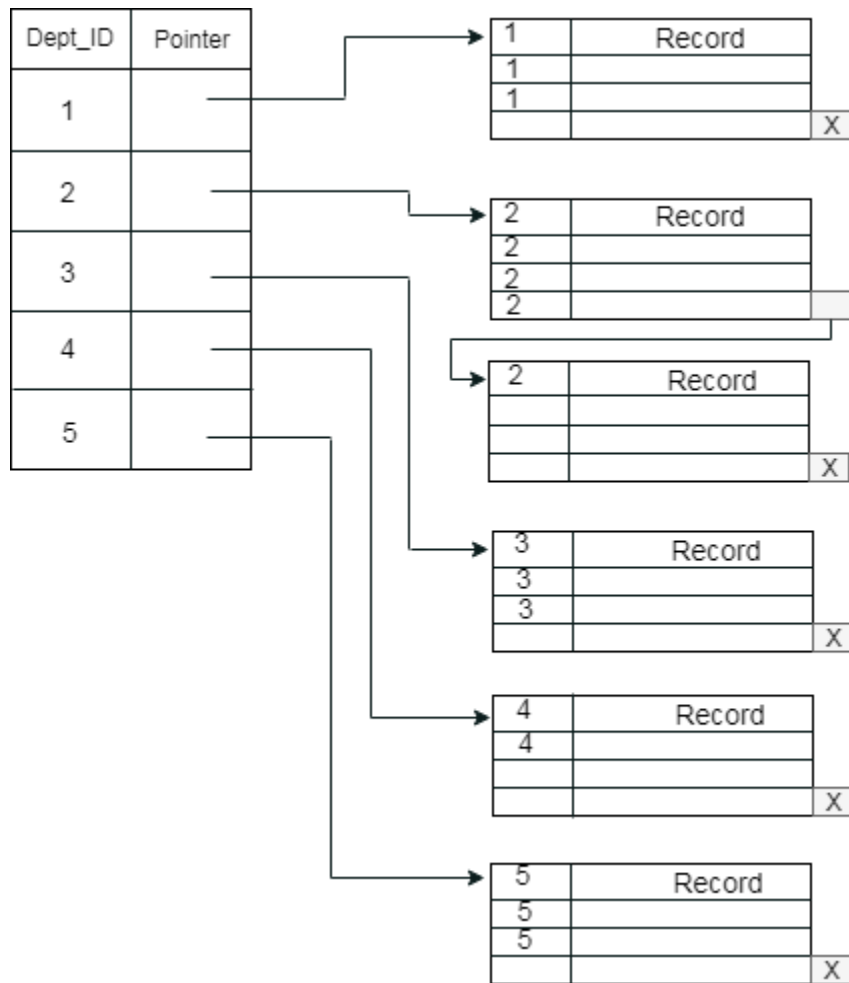
A clustered index determines the physical order of data in a table. In other words, the order in which the rows are stored on disk is the same as the order of the index key values. There can be only one clustered index on a table, but the table can have multiple non-clustered (or secondary) indexes.



Example: suppose a company contains several employees in each department. Suppose we use a clustering index, where all employees which belong to the same Dept_ID are considered within a single cluster, and index pointers point to the cluster as a whole. Here Dept_Id is a non-unique key.



The previous schema is little confusing because one disk block is shared by records which belong to the different cluster. If we use separate disk block for separate clusters, then it is called better technique.



Characteristics of a Clustered Index:

1. It dictates the physical storage order of the data in the table.
2. There can **only be one clustered index** per table.
3. It can be created on columns with **non-unique values**, but if it's on a non-unique column, the DBMS will usually add a uniqueifier to make each entry unique.
4. Lookup based on the clustered index is **fast** because the desired data is directly located without the need for additional lookups (unlike non-clustered indexes which require a second lookup to fetch the data).

Clustered Index Example

Imagine a `Books` table with the following records:

BookID	Title	Genre
3	A Tale of Two Cities	Fiction
1	Database Systems	Academic
4	Python Programming	Technical
2	The Great Gatsby	Fiction

If we create a clustered index on `BookID`, the physical order of records would be rearranged based on the ascending order of `BookID`.

The table would then look like this:

BookID	Title	Genre
1	Database Systems	Academic
2	The Great Gatsby	Fiction
3	A Tale of Two Cities	Fiction
4	Python Programming	Technical

Now, when you want to find a book based on its ID, the DBMS can quickly locate the data because the data is stored in the order of the BookID.

Benefits of a Clustered Index:

- **Fast Data Retrieval:** Because the data is stored sequentially in the order of the index key, range queries or ordered queries can be very efficient.
- **Data Pages:** With the data being stored sequentially, the number of data pages that need to be read from the disk is minimized.
- **No Additional Lookups:** Once the key is located using the index, there's no need for additional lookups to fetch the data, as it is stored right there.

Drawbacks of a Clustered Index:

- **Overhead on Inserts/Updates:** Because the data must be stored physically in the order of the index keys, inserts or updates can be slower since they might require data pages to be rearranged.
- **Single Clustered Index:** You can have only one clustered index per table, so you have to choose wisely based on the most critical queries' requirements.

In practice, it's often beneficial to have the primary key of a table also be the clustered index, but this is not a requirement. The decision of which column to use as a clustered index should be based on query patterns, the nature of the data, and the characteristics of the application using the database.

Primary Indexes

A primary index is an ordered file whose records are of fixed length with two fields. The first field is the same as the primary key of the data file, and the second field is a pointer to the data block where that specific key can be found.

The primary index can be classified into two types

1. Dense Index: In this, an index entry appears for every search key value in the data file.
2. Sparse (or Non-Dense) Index: Here, index records are created only for some of the search key values. A sparse index reduces the size of the index file.

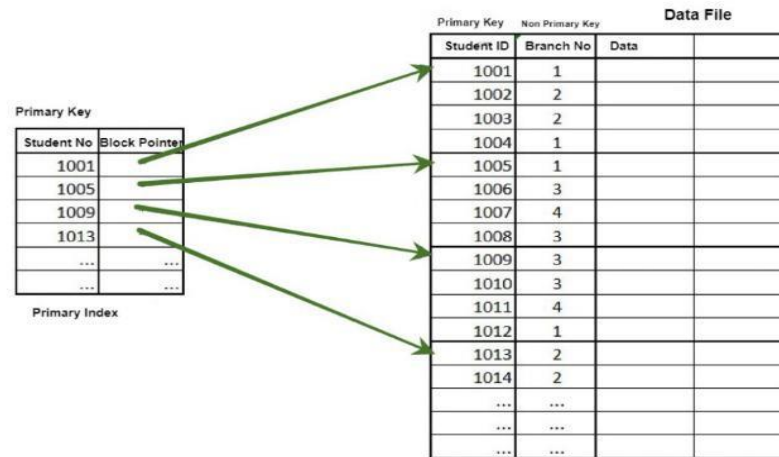
Characteristics of a Primary Index:

1. It is based on the primary key of the table.
2. The primary key must have unique values.
3. The index entries are sorted in the same order as the data file (hence often a clustered index).
4. The index has one entry for each block in a data file, not for each record.

Working of Primary Indexing

- In primary indexing, the data file is sorted or clustered based on the primary key as shown in the below figure.

- An **index file** (also known as the **index table**) is created alongside the data file.
- The index file contains pairs of primary key values and pointers to the corresponding data records.
- Each entry in the index file corresponds to a block or page in the data file.



Primary Index Example

Let's assume a simple table named `Students` that stores data about students. The table has the following structure:

| RollNumber (Primary Key) | Name | Age |

|-----|-----|-----|

| 1001 | Madhu | 20 |

| 1003 | Mahi | 22 |

| 1007 | Ramu | 21 |

| 1010 | Durga | 23 |

Assuming each block of our storage can hold two records, our blocks will be:

- Block 1: Contains records for RollNumbers 1001 and 1003.
- Block 2: Contains records for RollNumbers 1007 and 1010.

Dense Primary Index:

| Key (RollNumber) | Pointer to Block |

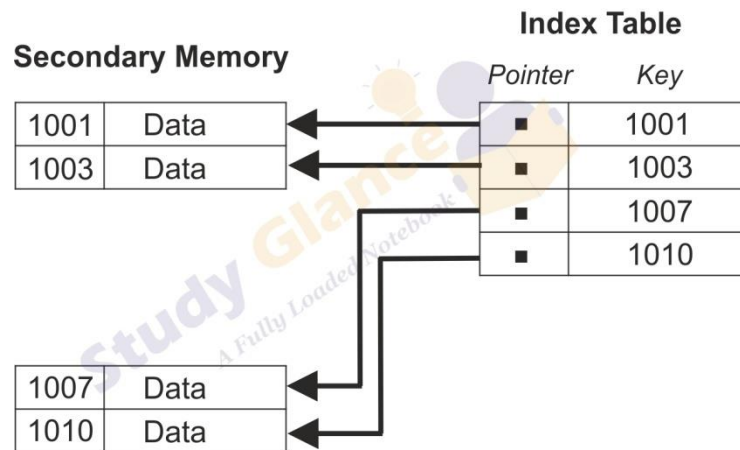
|-----|-----|

| 1001 | Block 1 |

| 1003 | Block 1 |

| 1007 | Block 2 |

| 1010 | Block 2 |



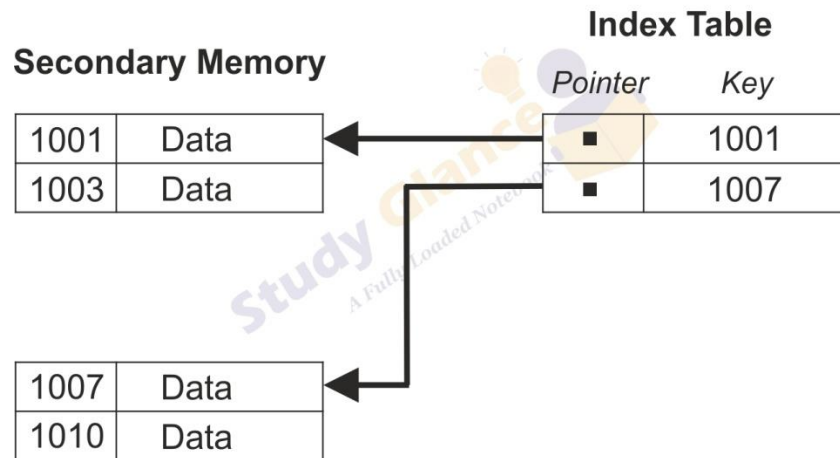
Sparse Primary Index:

| Key (RollNumber) | Pointer to Block |

|-----|-----|

| 1001 | Block 1 |

| 1007 | Block 2 |



In the dense primary index, there's an entry for every key value, whereas in the sparse primary index, only the first key value of each block (the block's anchor record) gets an entry in the index. In this way, sparse indexes are more compact but might require more sequential block reads if a search key isn't an anchor for a block.

Benefits of a Primary Index:

- Fast retrieval of records based on the primary key.
- Efficient for range-based queries due to sorted order.
- Can be used to enforce the uniqueness of the primary key.

However, while the primary index provides rapid access and ensures the uniqueness of records, it can be slower for insertions, updates, and deletions, as maintaining the sorted order of the index can be time-consuming.

Secondary Indexes

A secondary index provides a secondary means (other than the primary key) to access table data. Unlike primary indexes, secondary indexes aren't necessarily unique and aren't based on the primary key of a table. The records in a secondary index are stored separately from the data, and each index entry points to the corresponding data record. A table can have multiple secondary indexes.

Characteristics of a :

1. Provides an alternative path to access the data.
2. Can be either dense or sparse.
3. Allows for non-unique values.
4. Does not guarantee the order of records in the data file.
5. Unlike a primary (often clustered) index, a secondary index is typically a non-clustered index. This means the physical order of rows in a table is not the same as the index order.

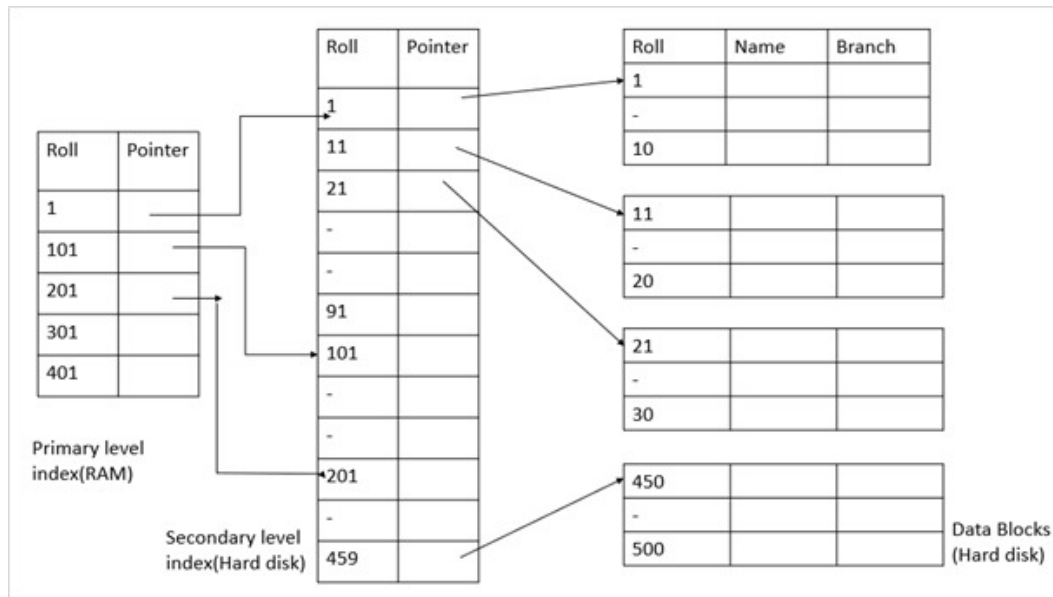
In secondary Index (Unique value) is created for each record in a data file which is a candidate key. Secondary index is a type of dense index and also called a non clustering index.

Secondary mapping size will be small as the two levels DB indexing is used.

- While creating the index, generally the index table is kept in the primary memory and the main table is kept in secondary memory because of its size.
- A table may contain thousands of records for this reason the sparse index becomes so large which cannot be handled in primary memory.
- Also, if we cannot keep the index in the primary memory then we lose the advantage of speed to access the data.
- For every large table, it is better to organize the index in multiple levels.

Example

Given below is an example of the secondary index –



Explanation

- **Step 1** – Here the primary level index which is created 100 records with a gap and therefore smaller in size, is kept in the RAM for quick reference.
- **Step 2** – If we need to find out the record of roll no 14, the index is first searched to find out the highest entry which is smaller than or equal to 14. We have 1,
- **Step 3** – The adjoining pointer leads us to the anchor record of the corresponding secondary level index, where another similar search is conducted.
- **Step 4** – This finally leads us to the actual data block whose anchor record is roll no 11.
- **Step 5** – we now come to roll no 11 where a sequential search is made to find out roll 14.

Multilevel index is a modification of the secondary level index. In this system we may use even more numbers of levels in case the table is even larger.

Benefits of a Secondary Index:

- Provides additional query paths, potentially speeding up query performance.
- Can be created on non-primary key columns, even on columns with non-unique values.
- Useful for optimizing specific query patterns.

Drawbacks of a Secondary Index:

- Increases the overhead of write operations since any insert, update, or delete operation on the table may require changes to the secondary index.
- Consumes additional storage space.
- May increase the complexity of database maintenance.

In a real-world scenario, database administrators and developers need to strike a balance. They need to identify which columns are frequently used in query conditions and might benefit from indexing, while also considering the trade-offs regarding space and write operation performance.

Index data Structures

One way to organize data entries is to hash data entries on the search key. Another way to organize data entries is to build a tree-like data structure that directs a search for data entries.

Hash-Based Indexing

In hash-based indexing, a hash function is used to convert a key into a hash code. This hash code serves as an index where the value associated with that key is stored. The goal is to distribute the keys uniformly across an array, so that access time is, on average, constant.

Let's break down some of these elements to further understand how hash-based indexing works in practice:

Buckets

In hash-based indexing, the data space is divided into a fixed number of slots known as "buckets." A bucket usually contains a single page (also known as a block), but it may have additional pages linked in a chain if the primary page becomes full. This is known as overflow.

Hash Function

The hash function is a mapping function that takes the search key as an input and returns the bucket number where the record should be located. Hash functions aim to distribute records uniformly across buckets to minimize the number of collisions (two different keys hashing to the same bucket).

Disk I/O Efficiency

Hash-based indexing is particularly efficient when it comes to disk I/O operations. Given a search key, the hash function quickly identifies the bucket (and thereby the disk page) where the desired record is located. This often requires only one or two disk I/Os, making the retrieval process very fast.

Insert Operations

When a new record is inserted into the dataset, its search key is hashed to find the appropriate bucket. If the primary page of the bucket is full, an additional overflow page is allocated and linked to the primary page. The new record is then stored on this overflow page.

Search Operations

To find a record with a specific search key, the hash function is applied to the search key to identify the bucket. All pages (primary and overflow) in that bucket are then examined to find the desired record.

Limitations

Hash-based indexing is not suitable for range queries or when the search key is not known. In such cases, a full scan of all pages is required, which is resource-intensive.

Hash-Based Indexing Example

Let's consider a simple example using employee names as the search key.

Employee Records

Name	Age	Salary
Alice	28	50000
Bob	35	60000
Carol	40	70000

Hash Function: $H(x) = \text{ASCII value of first letter of the name} \bmod 3$

- **Alice:** $65 \bmod 3 = 2$
- **Bob:** $66 \bmod 3 = 0$
- **Carol:** $67 \bmod 3 = 1$

Buckets:

Bucket 0: Bob
Bucket 1: Carol
Bucket 2: Alice

Pros of Hash-Based Indexing

- Extremely fast for exact match queries.
- Well-suited for equality comparisons.

Cons of Hash-Based Indexing

- Not suitable for range queries (e.g., "SELECT * FROM table WHERE age BETWEEN 20 AND 30").
- Performance can be severely affected by poor hash functions or a large number of collisions.

Tree-based Indexing

The most commonly used tree-based index structure is the B-Tree, and its variations like B+ Trees and B* Trees. In tree-based indexing, data is organized into a tree-like structure. Each node represents a range of key values, and leaf nodes contain the actual data or pointers to the data.

Why Tree-based Indexing?

Tree-based indexes like B-Trees offer a number of advantages:

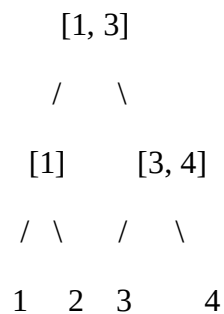
- **Sorted Data:** They maintain data in sorted order, making it easier to perform range queries.
- **Balanced Tree:** B-Trees and their variants are balanced, meaning the path from the root node to any leaf node is of the same length. This balancing ensures that data retrieval times are consistently fast, even as the dataset grows.
- **Multi-level Index:** Tree-based indexes can be multi-level, which helps to minimize the number of disk I/Os required to find an item.
- **Dynamic Nature:** B-Trees are dynamic, meaning they're good at inserting and deleting records without requiring full reorganization.
- **Versatility:** They are useful for both exact-match and range queries.

Tree-based Indexing Example

Continuing with the "Students" table:

ID	Name
1	Abhi
2	Bharath
3	Chinni
4	Devid

A simplified B-Tree index could look like this:



In the tree, navigating from the root to the leaf nodes will lead us to the desired data record.

Pros of Tree-based Indexing:

- Efficient for range queries.
- Good for both exact and partial matches.
- Keeps data sorted.

Cons of Tree-based Indexing:

- Slower than hash-based indexing for exact queries.
- More complex to implement and maintain.

Hash Based Indexing

For a huge database structure, it can be almost next to impossible to search all the index values through all its level and then reach the destination data block to retrieve the desired data. Hashing is an effective technique to calculate the direct location of a data record on the disk without using index structure.

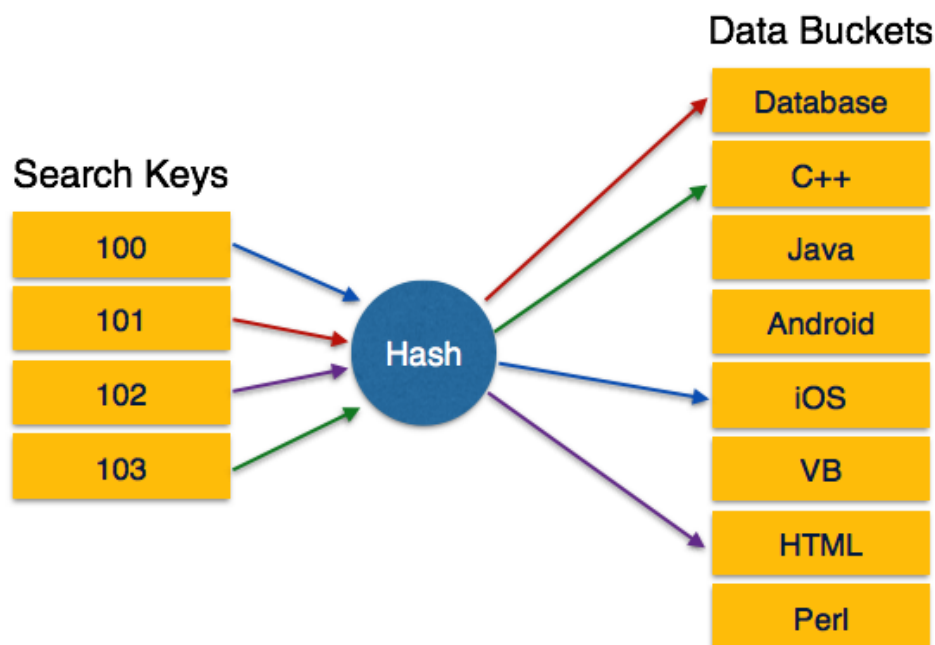
Hashing uses hash functions with search keys as parameters to generate the address of a data record.

Hash Organization

- **Bucket** – A hash file stores data in bucket format. Bucket is considered a unit of storage. A bucket typically stores one complete disk block, which in turn can store one or more records.
- **Hash Function** – A hash function, **h**, is a mapping function that maps all the set of search-keys **K** to the address where actual records are placed. It is a function from search keys to bucket addresses.

Static Hashing

In static hashing, when a search-key value is provided, the hash function always computes the same address. For example, if mod-4 hash function is used, then it shall generate only 5 values. The output address shall always be same for that function. The number of buckets provided remains unchanged at all times.



Operation

- **Insertion** – When a record is required to be entered using static hash, the hash function **h** computes the bucket address for search key **K**, where the record will be stored.

Bucket address = $h(K)$

- **Search** – When a record needs to be retrieved, the same hash function can be used to retrieve the address of the bucket where the data is stored.
- **Delete** – This is simply a search followed by a deletion operation.

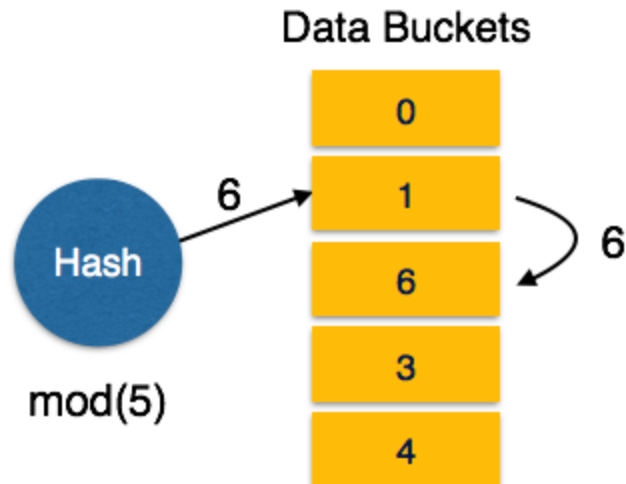
Bucket Overflow

The condition of bucket-overflow is known as **collision**. This is a fatal state for any static hash function. In this case, overflow chaining can be used.

- **Overflow Chaining** – When buckets are full, a new bucket is allocated for the same hash result and is linked after the previous one. This mechanism is called **Closed Hashing**.



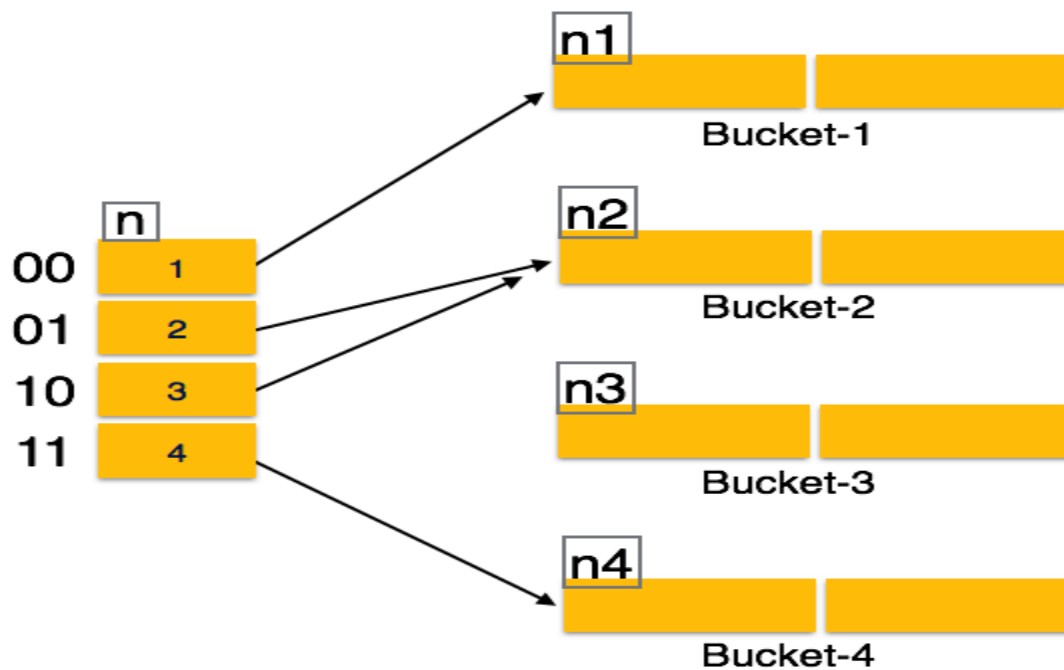
- **Linear Probing** – When a hash function generates an address at which data is already stored, the next free bucket is allocated to it. This mechanism is called **Open Hashing**.



Dynamic Hashing

The problem with static hashing is that it does not expand or shrink dynamically as the size of the database grows or shrinks. Dynamic hashing provides a mechanism in which data buckets are added and removed dynamically and on-demand. Dynamic hashing is also known as **extended hashing**.

Hash function, in dynamic hashing, is made to produce a large number of values and only a few are used initially.



Organization

The prefix of an entire hash value is taken as a hash index. Only a portion of the hash value is used for computing bucket addresses. Every hash index has a depth value to signify how many bits are used for computing a hash function. These bits can address 2^n buckets. When all these bits are consumed – that is, when all the buckets are full – then the depth value is increased linearly and twice the buckets are allocated.

Operation

- **Querying** – Look at the depth value of the hash index and use those bits to compute the bucket address.
- **Update** – Perform a query as above and update the data.
- **Deletion** – Perform a query to locate the desired data and delete the same.
- **Insertion** – Compute the address of the bucket
 - If the bucket is already full.
 - Add more buckets.
 - Add additional bits to the hash value.
 - Re-compute the hash function.
 - Else
 - Add data to the bucket,
 - If all the buckets are full, perform the remedies of static hashing.

Hashing is not favorable when the data is organized in some ordering and the queries require a range of data. When data is discrete and random, hash performs the best.

Hashing algorithms have high complexity than indexing. All hash operations are done in constant time.

Comparison of File Organizations

File organization, indexing, and performance tuning are three interconnected areas in the realm of Database Management Systems, each contributing to the overall efficiency and effectiveness of data storage and retrieval. Below is a comparison of these three concepts, focusing on their objectives, methodologies, and implications.

File Organizations

1. Objective: To physically store records on storage media in an organized manner.

2. Methodologies: Includes sequential, random (or direct), and hashed file organizations, among others.

3. Implications:

- Sequential organization is suitable for batch processing but inefficient for random access.
- Direct or random organization allows fast access but can be inefficient in terms of storage space.
- Hashed file organization is excellent for equality searches but not for range-based queries.

4. Real-world Examples: Ledger systems, log files, archival systems.

Indexing

1. Objective: To create a data structure that improves the speed of data retrieval operations.

2. Methodologies: Includes clustered, non-clustered, primary, secondary, composite, bitmap, and hash indexes, among others.

3. Implications:

- Clustered indexes are excellent for range-based queries but slow down insert/update operations.
- Non-clustered indexes improve data retrieval speed but can take up additional storage.
- Bitmap indexes are useful for low-cardinality columns.

4. Real-world Examples: Search engines, e-commerce websites, any application that requires fast data retrieval.

Points of Comparison on File organization and indexing

1. Granularity:

- File organization is about how data is stored at the file level.
- Indexing is about improving data access at the table or even column level.
- Performance tuning is a broad set of activities that can encompass both file organization and indexing among many other techniques.

2. Resource Usage:

- File organization techniques aim to use disk space efficiently.
- Indexing aims to use both disk space and memory for fast data retrieval.
- Performance tuning aims to optimize all system resources including CPU, memory, disk, and network bandwidth.

3. Query Efficiency:

- File organization generally impacts how efficiently data can be read or written to disk.
- Indexing significantly impacts how efficiently queries can retrieve data.
- Performance tuning seeks to optimize both read and write operations through a variety of methods.

4. Complexity:

- File organization is relatively straightforward.
- Indexing can become complex depending on the types of indexes and the nature of the queries.
- Performance tuning is usually the most complex as it involves a holistic understanding of hardware, software, data, and queries.

Intuitions for tree Indexes

Consider a file of Students records sorted by *gpa*. To answer a range selection such as "Find all students with a *gpa* higher than 3.0," we must identify the first such student by doing a binary search of the file and then scan the file from that point on. If the file is large, the initial binary search can be quite expensive, since cost is proportional to the number of pages fetched; can we improve upon this method'?

whole idea is to create a second file with whole record per page in the original (data) file, of the form (*first key on page, pointer to page*), again sorted by the key attribute (which is *gpa* in our example). The format of a page in the second *index* file is illustrated in Figure 10.1.

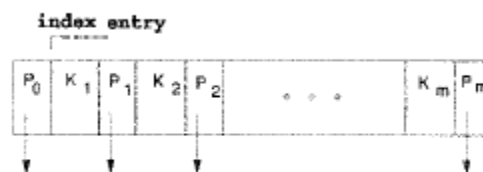


Figure 10.1 Format of an Index Page

We refer to pairs of the form (*key, pointer*) as index *entries* or just *entries* when the context is clear. Note that each index page contains whole pointer more than the number of keys---each key serves as a *separator*- for the contents of the pages pointed to by the pointers to its left and right.

The simple index file data structure is illustrated in Figure 10.2.

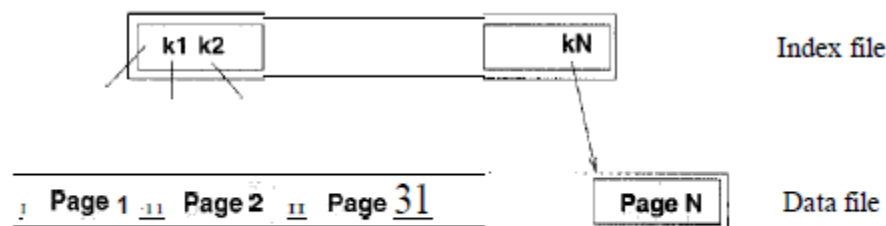


Figure 10.2 One-Level Index Structure

We can do a binary search of the index file to identify the page containing the first key (*gpa*.) value that satisfies the range selection (in our example, the first student with *gpa*. over 3.0) and follow the pointer to the page containing the first data. record with that key value. We can then scan the data file sequentially from that point on to retrieve other qualifying records. This example uses the index to find the first data page containing a Students record with *gpa*. Greater than 3.0, and the data file is scanned from that point on to retrieve other such Students records.

Because the size of an entry in the index file (key value and page id) is likely to be much smaller than the size of a page, and only one such entry exists per page of the data file, the index file is likely to be much smaller than the data file; therefore, a binary search of the index file is much faster than a binary search of the data file. However, a binary search of the index file could still be fairly expensive, and the index file is typically still large enough to make inserts and deletes expensive.

The potential large size of the index file motivates the tree indexing idea: Why not apply the previous step of building an auxiliary structure all the collection of *index* records and so on recursively until the smallest auxiliary structure fits whole one page? This repeated construction of a one-level index leads to a tree structure with several levels of non-leaf pages.

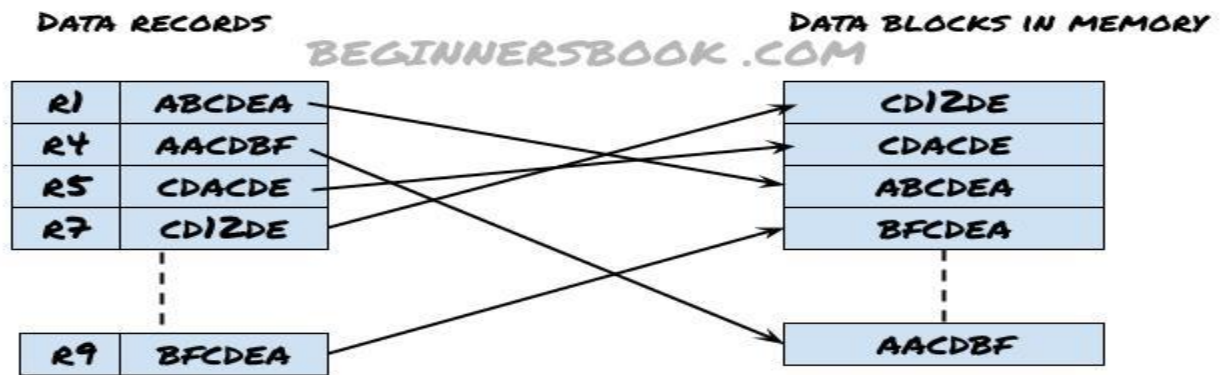
The power of the approach comes from the fact that locating a record (given a search key value) involves a traversal from the root to a leaf, with one I/O (at most; some pages, e.g.) the root, are likely to be in the buffer pool) per level. Given the typical fan-out value (over 100), trees rarely have more than 3-4 levels.

The next issue to consider is how the tree structure can handle inserts and deletes of data entries. Two distinct approaches have been used, leading to the ISAM and B+ tree data structures, which we discuss in subsequent sections.

Indexed sequential access method (ISAM)

Indexed sequential access method also known as ISAM method, is an upgrade to the conventional sequential file organization method. You can say that it is an advanced version of sequential file organization method. In this method, primary key of the record is stored with an address, this address is mapped to an address of a data block in memory. This address field works as an index of the file.

In this method, reading and fetching a record is done using the index of the file. Index field contains the address of a data record in memory, which can be quickly used to read and fetch the record from memory.



ISAM FILE ORGANIZATION IN DBMS

Advantages of ISAM

1. **Searching a record is faster** in ISAM file organization compared to other file organization methods as the primary key can be used to identify the record and since primary key also has the address of the record, it can read and fetch the data from memory.
2. This method is **more flexible compared to other methods** as this **allows to generate the index field (address field) for any column of the record**. This makes searching easier and efficient as searches can be done using multiple column fields.
3. This allows **range retrieval of the records** since the address file is stored with the primary key of the record, we can retrieve the record based on a certain range of primary key columns.
4. **This method allow partial searches as well**. For example, employee name starting with “St” can be used to search all the employees with the name starting with letters “St”. This will result all the records where employee name begins with the letters “St”.

Disadvantages of ISAM

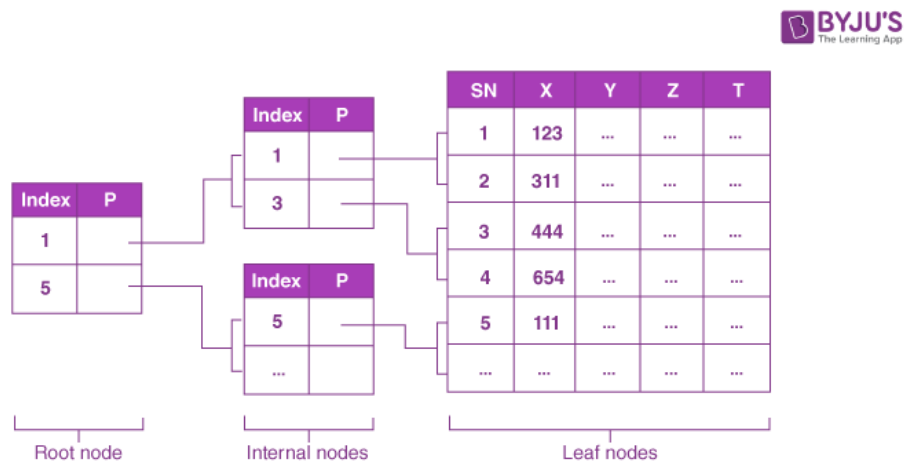
1. Requires **additional space** in the memory to store the index field.
2. After adding a record to the file, the file needs to be re-organized to maintain the sequence based on primary key column.
3. **Requires memory cleanup** because when a record is deleted, the space used by the record needs to be released in order to be used by the other record.
4. **Performance issues** are there if there are frequent deletion of records, as every deletion needs a memory cleanup and optimization.

B+ Tree

What is B+ Tree in DBMS?

A balanced binary search tree is the B+ Tree. It uses a multilevel indexing system. Leaf nodes in the B plus tree represent actual data references. The B plus tree keeps all of the leaf nodes at the same height. A link list is used to connect the leaf nodes in the B+ Tree. As a result, a B+ Tree can allow both random and sequential access.

Multilevel indexing is a crucial topic to grasp before understanding B+ Tree in the data structure. The index of indices is formed in multilevel indexing, as shown in the diagram below. It facilitates and accelerates data access.



B+ Tree Properties

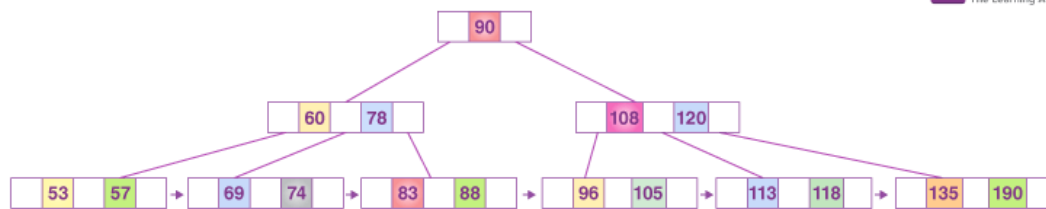
- The leaves are all at the same height.
- There are at least two children in the root.
- Except for root, each node can have a max of m children and a min of $m/2$ children.
- A max of $m - 1$ keys and a min of $m/2 - 1$ keys can be stored in each node.

Both keys and records can be placed in the internal and leaf nodes of a [B Tree](#). In a B+ Tree, records or data can only be kept on the leaf nodes, whereas key values can only be placed on the internal nodes.

To make search queries more efficient, the leaf nodes of the B plus tree in the data structure are connected together in the form of singly-linked lists.

B+ Trees are used to store vast amounts of data that are too large to fit in the main memory. The internal nodes of the B+ Tree (the keys to access records) are securely stored memory, whereas leaf nodes are placed in the secondary memory due to the restricted amount of main memory.

B plus tree internal nodes are often referred to as index nodes. The following diagram depicts a B+ Tree of order 3.

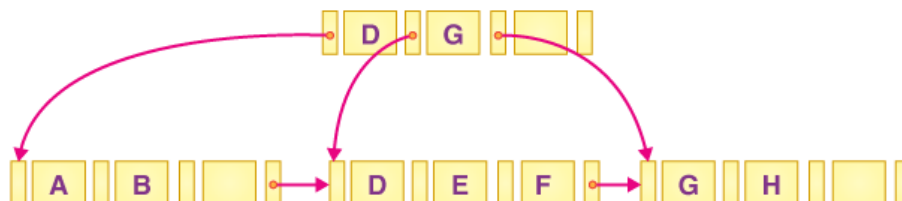


Pros of B+ Tree

- In an equal number of disc accesses, records can be retrieved.
- In comparison to the B tree, the B+ Tree's height stays balanced and is lower.
- The stored data in a B+ Tree can be accessed both sequentially and directly.
- Indexing is done via keys.
- Because the data is only stored on the leaf nodes, search queries are faster.

B+ Tree Structure

- Every leaf node in the B+ Tree is at the same distance from the root node. The B+ Tree has an order of n , with n being the same for all B plus trees.
- It has a leaf node and an internal node.



Internal Node

- Except for the root node, an internal node present in the B+ Tree can at least have $n/2$ record pointers.
- The internal node of the tree can only have n pointers.

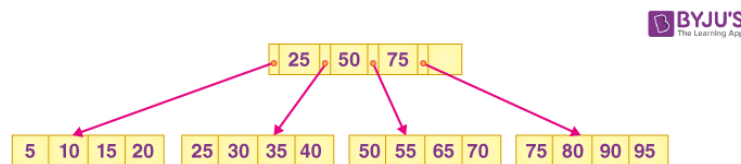
Leaf Node

- At least $n/2$ key values and $n/2$ record pointers can be found in the B plus tree's leaf node.
- A leaf node can only have n record pointers and n key values.
- Every B+ Tree leaf node has a block pointer P that points to the next leaf node.

Searching Records in B+ Tree

Suppose that we need to find 55 in the B+ Tree structure below. We'll start by fetching for the intermediary node, which will lead to the leaf node, which may have a record of 55.

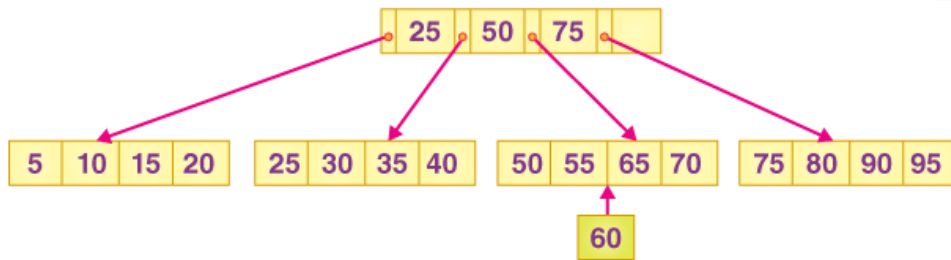
So we'll identify a branch between 50 to 75 nodes in the intermediary node. Then we'll be redirected to the 3rd leaf node at the conclusion. In this case, the DBMS will use a sequential search to locate 55.



Insertion in B+ Tree

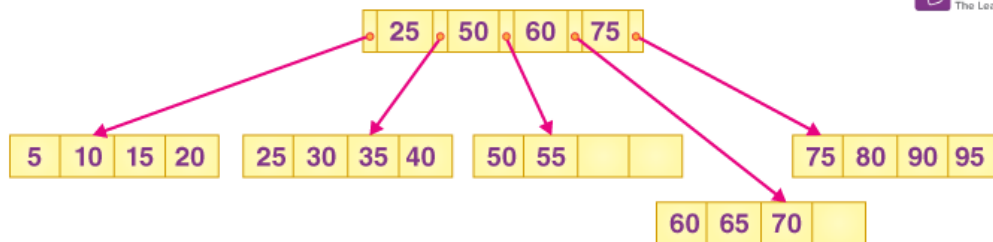
Suppose that we wish to add a record 60 to the structure below. After 55, it will reach the third leaf node. It's a balanced tree, and one of its leaf nodes is already full; thus, we can't put 60 in there.

We must split the leaf node in this scenario so that it may be added to the tree without impacting the fill factor, balance or order.



The values of the 3rd leaf node are (50, 55, 60, 65, 70), and the current root node is 50. To keep the tree's equilibrium, we'll divide the leaf node in half in the middle. As a result, (50, 55) & (60, 65, 70) can be grouped into two leaf nodes.

The intermediate node can't branch from 50 if these two must be leaf nodes. It must have 60 added to it, and thereafter links to a new leaf node will be available.

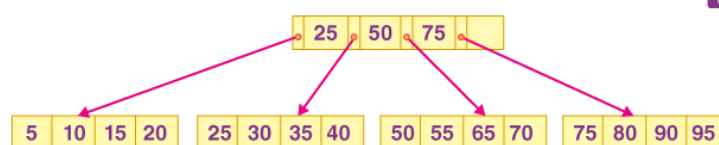


When there is an overflow, we can enter an entry in this manner. In a typical scenario, finding the node in which it fits and then placing it in that leaf node is extremely simple.

Deletion in B+ Tree

Imagine that we want to remove 60 from the previous example. In this scenario, we must subtract 60 from both the intermediate node and the fourth leaf node. The tree will not satisfy the B plus tree rule if it is removed from the intermediate node. As a result, we must alter it in order to achieve a balanced tree.

It will look like the following after eliminating node 60 from the B+ Tree and rearranging the nodes:



B Tree VS B+ Tree

SN	B Tree	B+ Tree
1	Search keys can not be repeatedly stored.	Redundant search keys can be present.
2	Data can be stored in leaf nodes as well as internal nodes	Data can only be stored on the leaf nodes.
3	Searching for some data is a slower process since data can be found on internal nodes as well as on the leaf nodes.	Searching is comparatively faster as data can only be found on the leaf nodes.
4	Deletion of internal nodes are so complicated and time consuming.	Deletion will never be a complexed process since element will always be deleted from the leaf nodes.
5	Leaf nodes can not be linked together.	Leaf nodes are linked together to make the search operations more efficient.

Dynamic Index Structure

what is A Dynamic Index Structure

A dynamic index structure is an index that automatically adjusts its structure as the underlying data grows, shrinks, or otherwise changes. Unlike static index structures, which are fixed in size and organization, dynamic index structures reorganize themselves in response to changes in the data they index. This makes them more flexible and scalable, capable of handling varying workloads and data distributions efficiently.

Dynamic index structures are crucial for [databases](#) that are expected to evolve over time, especially when it comes to insertion, deletion, and updating of records. They maintain balanced tree structures, self-adjust, and optimize, ensuring that the [database](#) operations remain efficient even as the data grows or changes.

Examples of Dynamic Index Structures:

B-Trees and B⁺-Trees: These are perhaps the most well-known examples of dynamic index structures. B-Trees and B⁺-Trees automatically split or merge nodes to maintain a balanced tree, ensuring that search, insert, and delete operations take logarithmic time. They are widely used in various types of [databases](#) and filesystems.

B⁺ Trees

A B⁺ Tree is a type of self-balancing tree structure commonly used in [databases](#) and file systems to maintain sorted data in a way that allows for efficient insertion, deletion, and search operations. B⁺ Trees are an extension of B-Trees but differ mainly in the way they handle leaf nodes, which contain all the key values and point to the actual records.

A B⁺ Tree of order n has the following properties:

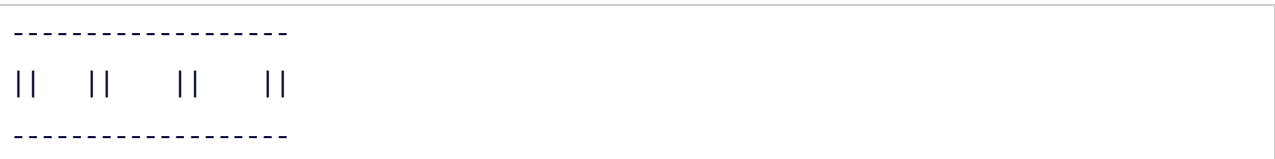
1. Every node has a maximum of n children.
2. Every node (except the root) has a minimum of $n/2$ children.
3. The tree is perfectly balanced, meaning that all leaf nodes are at the same level.
4. All keys are stored in the leaf nodes, and the internal nodes act as 'guides' to locate the leaf nodes faster.

Operations on B⁺ Trees:

1. Search: Starts at the root and traverses down the tree, guided by the key values in each node, until it reaches the appropriate leaf node.
2. Insert: Inserts a new key-value pair and then reorganizes the tree as needed to maintain its properties.
3. Delete: Removes a key-value pair and then reorganizes the tree, again to maintain its properties.

Example of B⁺ Tree Operations

Let's say we have a B⁺ Tree of order 4, and we want to insert the keys $[10, 20, 5, 6, 12, 30, 7, 17]$ into an initially empty tree.



Insertion

1. Insert 10:

- The tree is empty, so 10 becomes the root.

[10]

2. Insert 20:

- There's room in the leaf node for 20.

[10, 20]

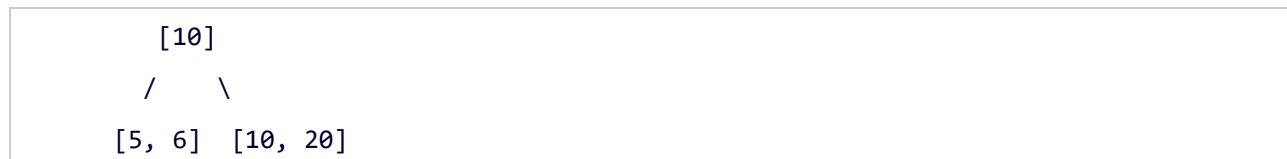
3. Insert 5:

- Still room in the leaf node for 5.

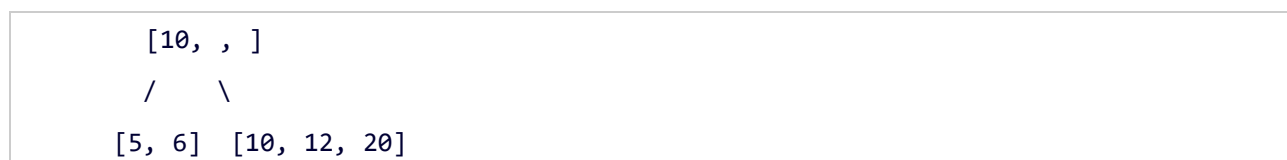
[5, 10, 20]

4. Insert 6:

- The leaf node is full; split it and promote the smallest key in the right node to be the new root.

**5. Insert 12:**

- Insert into the appropriate leaf node.

**6. Insert 30:**

- Need to split the right leaf node, promote 20.

```

      [10 , 20 , ]
     /  |  \
    [5, 6] [10, 12] [20, 30]

```

7. Insert 7:

- Insert into the appropriate leaf node.

```

      [10 , 20 , ]
     /  |  \
    [5, 6, 7] [10, 12] [20, 30]

```

8. Insert 17:

- Insert into the appropriate leaf node and split.

```

      [10 , 20 , ]
     /  |  \
    [5, 6, 7] [10, 12] [17, 20, 30]

```

- Here, the middle internal node gets split, and 17 is promoted.

```

      [10 , 17 , 20 ]
     /  |  |  \
    [5, 6, 7] [10, 12] [17] [20, 30]

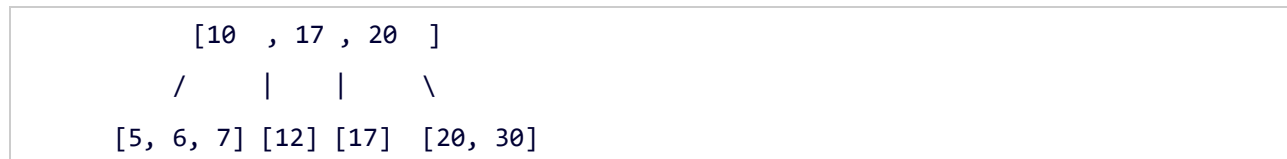
```

Search (for 12):

- Start at the root, go down the second child because $12 > 10$ and $12 < 17$, and find 12 in the corresponding leaf node.

Deletion (of 10):

1. Remove 10 from the leaf node.



2. Since the key 10 is also present in the internal node, we replace it with its in-order predecessor (or successor based on design), which is 7.

